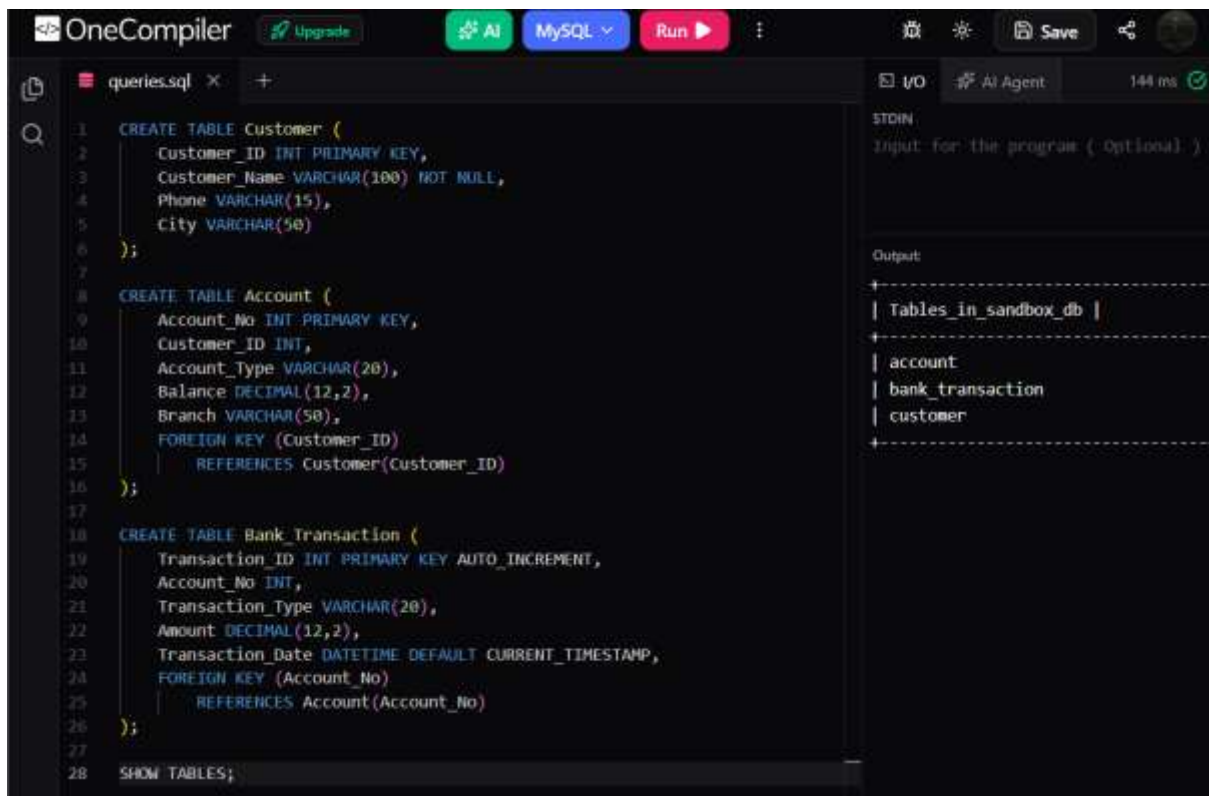


BANK DATABASE – ACID PROPERTIES & ISOLATION

1. Database and Table Creation



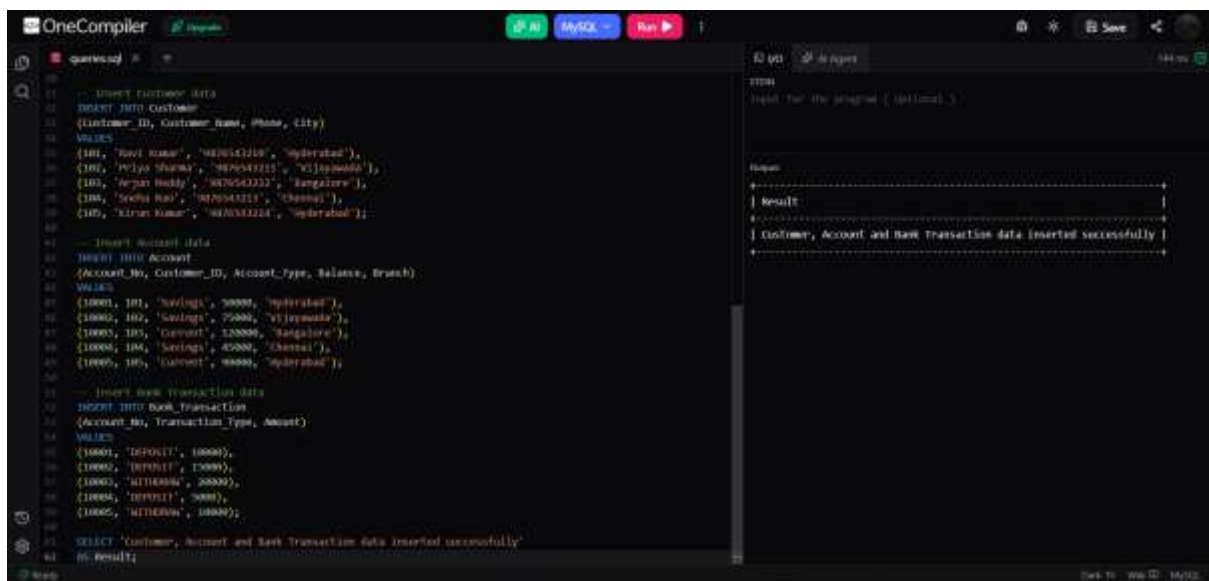
The screenshot shows the OneCompiler IDE interface. The main editor displays a MySQL script for creating a database schema. The script includes three table creation statements: 'Customer', 'Account', and 'Bank Transaction'. The 'Customer' table has columns for Customer_ID (primary key), Customer_Name, Phone, and City. The 'Account' table has columns for Account_No (primary key), Customer_ID (foreign key to Customer), Account_Type, Balance, and Branch. The 'Bank Transaction' table has columns for Transaction_ID (primary key, auto-increment), Account_No (foreign key to Account), Transaction_Type, Amount, and Transaction_Date (default current timestamp). The script concludes with a 'SHOW TABLES;' command. On the right side, the 'I/O' panel shows the execution output, which lists the tables created in the 'sandbox_db' database: 'account', 'bank_transaction', and 'customer'.

```
1 CREATE TABLE Customer (  
2   Customer_ID INT PRIMARY KEY,  
3   Customer_Name VARCHAR(100) NOT NULL,  
4   Phone VARCHAR(15),  
5   City VARCHAR(50)  
6 );  
7  
8 CREATE TABLE Account (  
9   Account_No INT PRIMARY KEY,  
10  Customer_ID INT,  
11  Account_Type VARCHAR(20),  
12  Balance DECIMAL(12,2),  
13  Branch VARCHAR(50),  
14  FOREIGN KEY (Customer_ID)  
15  | REFERENCES Customer(Customer_ID)  
16 );  
17  
18 CREATE TABLE Bank Transaction (  
19  Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,  
20  Account_No INT,  
21  Transaction_Type VARCHAR(20),  
22  Amount DECIMAL(12,2),  
23  Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,  
24  FOREIGN KEY (Account_No)  
25  | REFERENCES Account(Account_No)  
26 );  
27  
28 SHOW TABLES;
```

Output:

```
+-----+  
| Tables_in_sandbox_db |  
+-----+  
| account  
| bank_transaction  
| customer  
+-----+
```

2. Insert Customer, Account and Bank Transaction Data

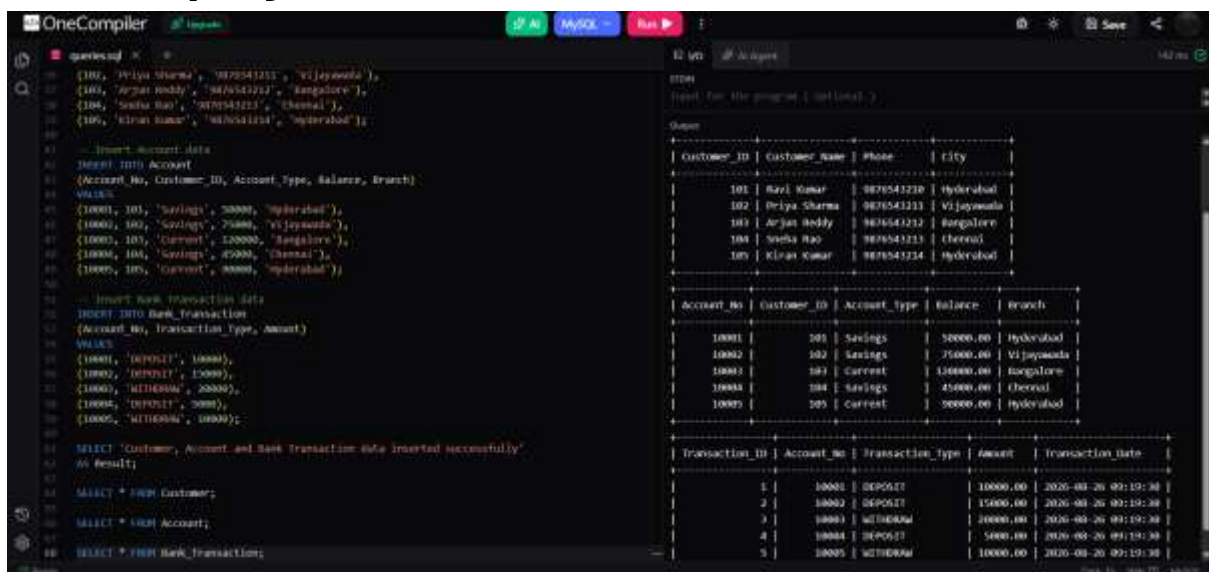


```
11 -- Insert Customer data
12 INSERT INTO Customer
13 (Customer_ID, Customer_Name, Phone, City)
14 VALUES
15 (101, 'Navi Kumar', '9876543210', 'Hyderabad'),
16 (102, 'Priya Sharma', '9876543211', 'Vijayawada'),
17 (103, 'Arjun Reddy', '9876543212', 'Bangalore'),
18 (104, 'Sneha Rao', '9876543213', 'Chennai'),
19 (105, 'Kiran Kumar', '9876543214', 'Hyderabad');
20
21 -- Insert Account data
22 INSERT INTO Account
23 (Account_No, Customer_ID, Account_Type, Balance, Branch)
24 VALUES
25 (10001, 101, 'Savings', 50000, 'Hyderabad'),
26 (10002, 102, 'Savings', 75000, 'Vijayawada'),
27 (10003, 103, 'Current', 120000, 'Bangalore'),
28 (10004, 104, 'Savings', 45000, 'Chennai'),
29 (10005, 105, 'Current', 90000, 'Hyderabad');
30
31 -- Insert Bank Transaction data
32 INSERT INTO Bank_Transaction
33 (Account_No, Transaction_Type, Amount)
34 VALUES
35 (10001, 'DEPOSIT', 10000),
36 (10002, 'DEPOSIT', 15000),
37 (10003, 'WITHDRAW', 20000),
38 (10004, 'DEPOSIT', 5000),
39 (10005, 'WITHDRAW', 10000);
40
41 SELECT 'Customer, Account and Bank Transaction data inserted successfully'
42 AS Result;
```

Output:

```
[ Result
----- ]
[ Customer, Account and Bank Transaction data inserted successfully ]
```

3 — Display All Bank Tables



```
43 -- Insert Customer data
44 INSERT INTO Customer
45 (Customer_ID, Customer_Name, Phone, City)
46 VALUES
47 (101, 'Navi Kumar', '9876543210', 'Hyderabad'),
48 (102, 'Priya Sharma', '9876543211', 'Vijayawada'),
49 (103, 'Arjun Reddy', '9876543212', 'Bangalore'),
50 (104, 'Sneha Rao', '9876543213', 'Chennai'),
51 (105, 'Kiran Kumar', '9876543214', 'Hyderabad');
52
53 -- Insert Account data
54 INSERT INTO Account
55 (Account_No, Customer_ID, Account_Type, Balance, Branch)
56 VALUES
57 (10001, 101, 'Savings', 50000, 'Hyderabad'),
58 (10002, 102, 'Savings', 75000, 'Vijayawada'),
59 (10003, 103, 'Current', 120000, 'Bangalore'),
60 (10004, 104, 'Savings', 45000, 'Chennai'),
61 (10005, 105, 'Current', 90000, 'Hyderabad');
62
63 -- Insert Bank Transaction data
64 INSERT INTO Bank_Transaction
65 (Account_No, Transaction_Type, Amount)
66 VALUES
67 (10001, 'DEPOSIT', 10000),
68 (10002, 'DEPOSIT', 15000),
69 (10003, 'WITHDRAW', 20000),
70 (10004, 'DEPOSIT', 5000),
71 (10005, 'WITHDRAW', 10000);
72
73 SELECT 'Customer, Account and Bank Transaction data inserted successfully'
74 AS Result;
75
76 SELECT * FROM Customer;
77
78 SELECT * FROM Account;
79
80 SELECT * FROM Bank_Transaction;
```

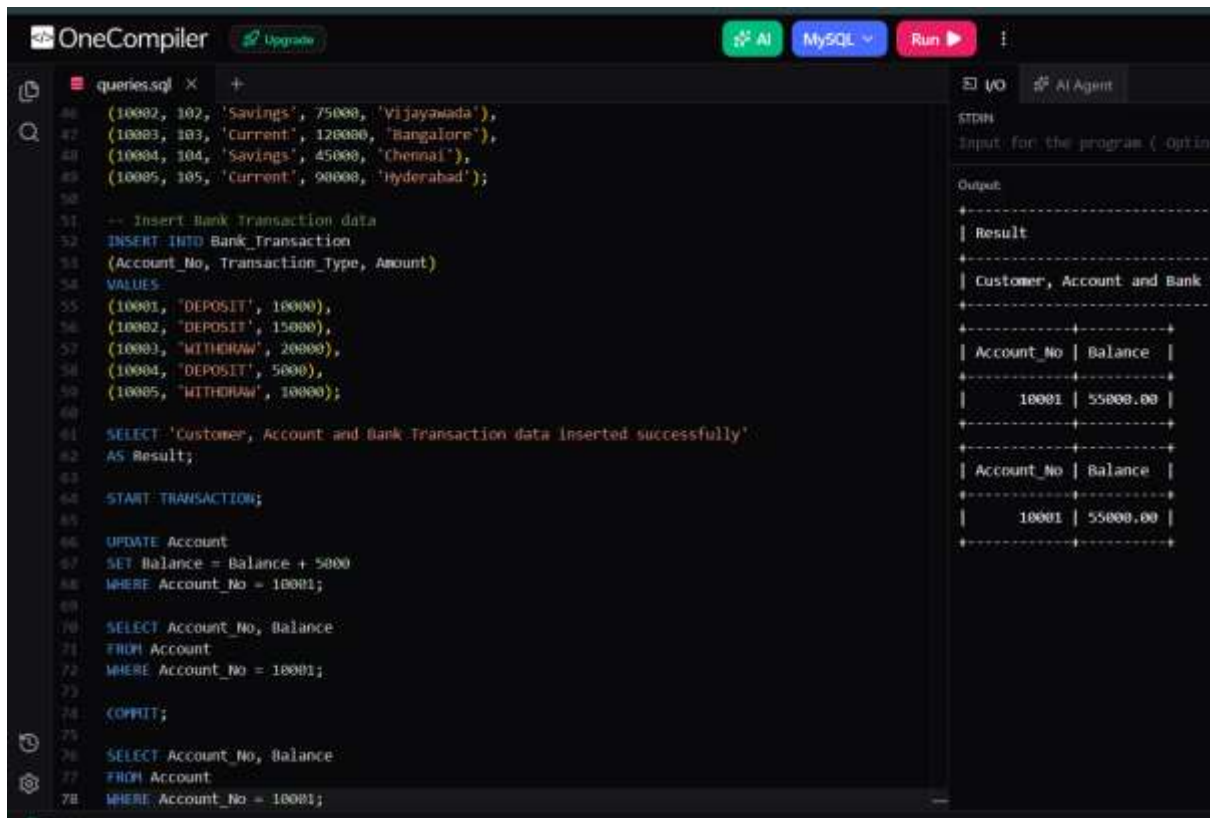
Output:

Customer_ID	Customer_Name	Phone	City
101	Navi Kumar	9876543210	Hyderabad
102	Priya Sharma	9876543211	Vijayawada
103	Arjun Reddy	9876543212	Bangalore
104	Sneha Rao	9876543213	Chennai
105	Kiran Kumar	9876543214	Hyderabad

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad
10002	102	Savings	75000.00	Vijayawada
10003	103	Current	120000.00	Bangalore
10004	104	Savings	45000.00	Chennai
10005	105	Current	90000.00	Hyderabad

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction Date
1	10001	DEPOSIT	10000.00	2020-08-26 09:19:30
2	10002	DEPOSIT	15000.00	2020-08-26 09:19:30
3	10003	WITHDRAW	20000.00	2020-08-26 09:19:30
4	10004	DEPOSIT	5000.00	2020-08-26 09:19:30
5	10005	WITHDRAW	10000.00	2020-08-26 09:19:30

4 — COMMIT: Deposit Transaction



The screenshot shows the OneCompiler MySQL IDE interface. The left pane displays a file named `queries.sql` with the following SQL code:

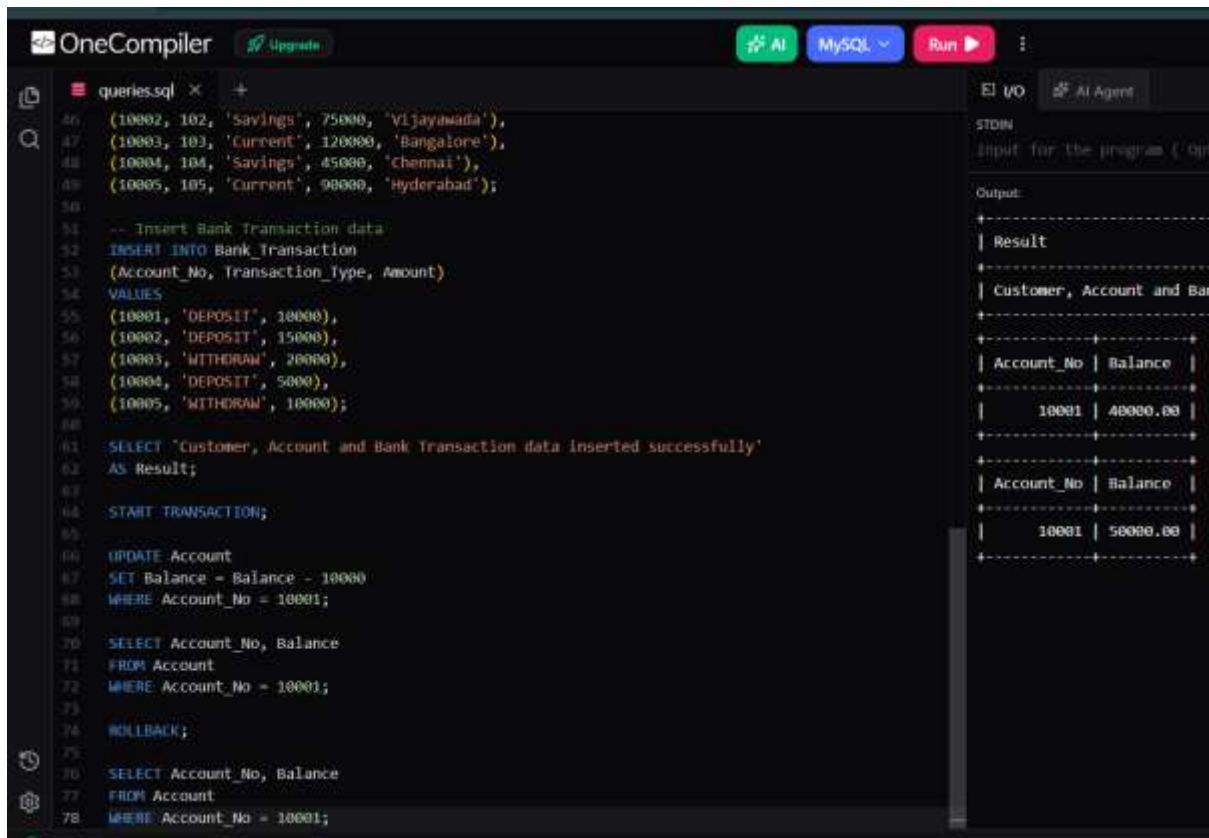
```
40 (10002, 102, 'Savings', 75000, 'Vijayawada'),
41 (10003, 103, 'Current', 120000, 'Bangalore'),
42 (10004, 104, 'Savings', 45000, 'Chennai'),
43 (10005, 105, 'Current', 90000, 'Hyderabad');
44
45 -- Insert Bank Transaction data
46 INSERT INTO Bank_Transaction
47 (Account_No, Transaction_Type, Amount)
48 VALUES
49 (10001, 'DEPOSIT', 10000),
50 (10002, 'DEPOSIT', 15000),
51 (10003, 'WITHDRAW', 20000),
52 (10004, 'DEPOSIT', 5000),
53 (10005, 'WITHDRAW', 10000);
54
55 SELECT 'Customer, Account and Bank Transaction data inserted successfully'
56 AS Result;
57
58 START TRANSACTION;
59
60 UPDATE Account
61 SET Balance = Balance + 5000
62 WHERE Account_No = 10001;
63
64 SELECT Account_No, Balance
65 FROM Account
66 WHERE Account_No = 10001;
67
68 COMMIT;
69
70 SELECT Account_No, Balance
71 FROM Account
72 WHERE Account_No = 10001;
```

The right pane shows the output of the queries. It includes a section for the first query's result:

Account_No	Balance
10001	55000.00

Below this, there is a section for the second query's result, which shows the same table structure but with no data rows displayed.

5 — ROLLBACK: Cancel Withdrawal



The screenshot shows the OneCompiler MySQL IDE. The editor contains a SQL script with the following lines:

```
46 (10002, 102, 'Savings', 75000, 'Vijayawada'),
47 (10003, 103, 'Current', 120000, 'Bangalore'),
48 (10004, 104, 'Savings', 45000, 'Chennai'),
49 (10005, 105, 'Current', 90000, 'Hyderabad');
50
51 -- Insert Bank Transaction data
52 INSERT INTO Bank Transaction
53 (Account_No, Transaction_Type, Amount)
54 VALUES
55 (10001, 'DEPOSIT', 10000),
56 (10002, 'DEPOSIT', 15000),
57 (10003, 'WITHDRAW', 20000),
58 (10004, 'DEPOSIT', 5000),
59 (10005, 'WITHDRAW', 10000);
60
61 SELECT 'Customer, Account and Bank Transaction data inserted successfully'
62 AS Result;
63
64 START TRANSACTION;
65
66 UPDATE Account
67 SET Balance = Balance - 10000
68 WHERE Account_No = 10001;
69
70 SELECT Account_No, Balance
71 FROM Account
72 WHERE Account_No = 10001;
73
74 ROLLBACK;
75
76 SELECT Account_No, Balance
77 FROM Account
78 WHERE Account_No = 10001;
```

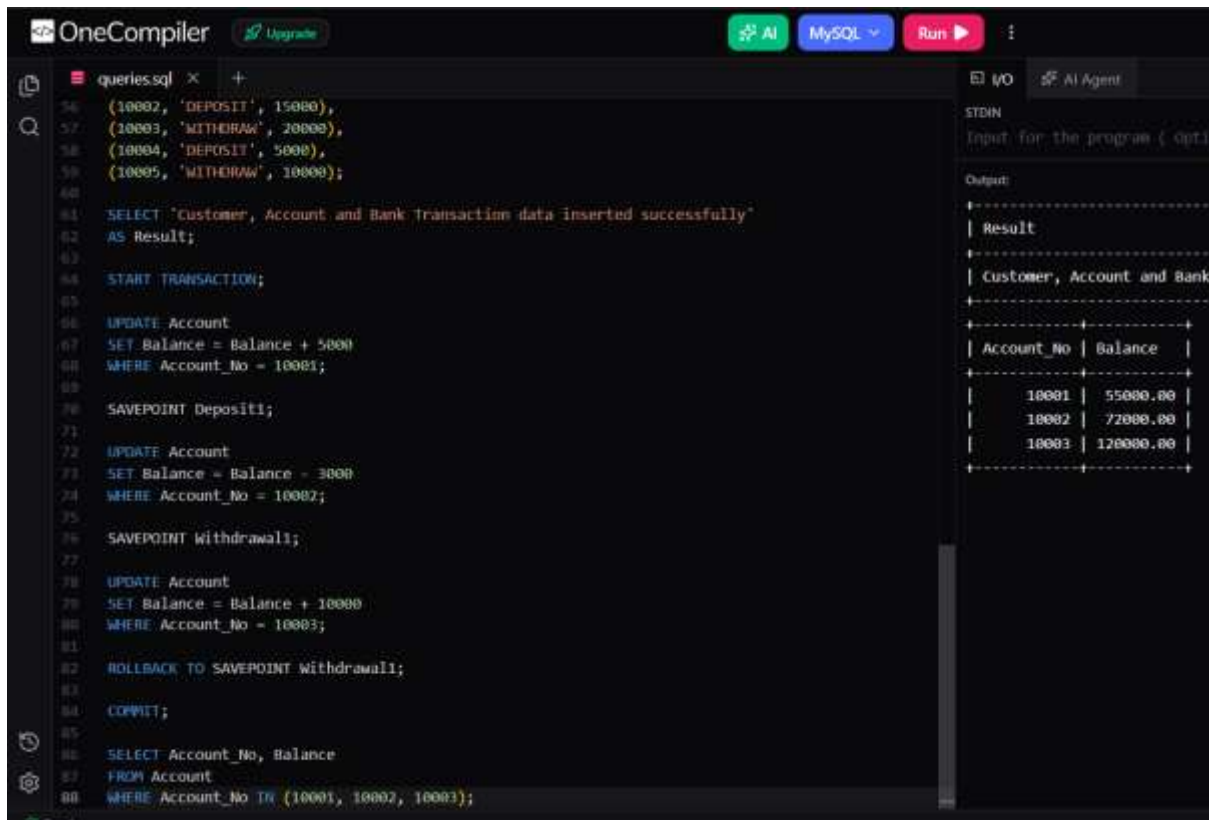
The output pane on the right shows the results of the queries:

STDIN
Input for the program (Optional):

Output:

```
+-----+
| Result |
+-----+
| Customer, Account and Bank Transaction data inserted successfully |
+-----+
| Account_No | Balance |
+-----+
| 10001 | 40000.00 |
+-----+
| Account_No | Balance |
+-----+
| 10001 | 50000.00 |
+-----+
```

6 — SAVEPOINT: Partial Rollback



The screenshot shows the OneCompiler MySQL IDE. The editor contains a SQL script with the following lines:

```
56: (10002, 'DEPOSIT', 15000),
57: (10003, 'WITHDRAW', 20000),
58: (10004, 'DEPOSIT', 5000),
59: (10005, 'WITHDRAW', 10000);
60:
61: SELECT 'Customer, Account and Bank Transaction data inserted successfully'
62: AS Result;
63:
64: START TRANSACTION;
65:
66: UPDATE Account
67: SET Balance = Balance + 5000
68: WHERE Account_No = 10001;
69:
70: SAVEPOINT Deposit1;
71:
72: UPDATE Account
73: SET Balance = Balance - 3000
74: WHERE Account_No = 10002;
75:
76: SAVEPOINT Withdrawal1;
77:
78: UPDATE Account
79: SET Balance = Balance + 10000
80: WHERE Account_No = 10003;
81:
82: ROLLBACK TO SAVEPOINT Withdrawal1;
83:
84: COMMIT;
85:
86: SELECT Account_No, Balance
87: FROM Account
88: WHERE Account_No IN (10001, 10002, 10003);
```

The output panel on the right shows the result of the query:

STDIN
Input for the program (opt1)

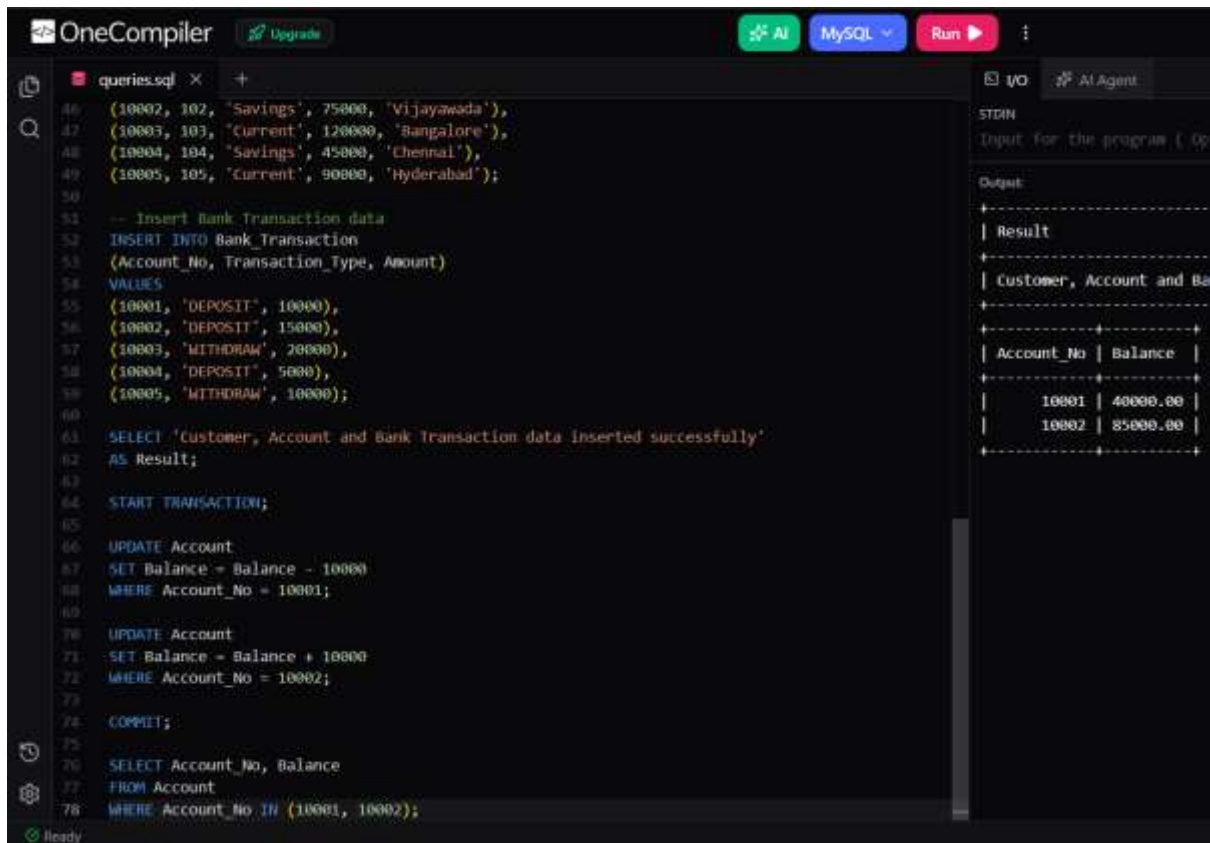
Output:

Result

Customer, Account and Bank

Account_No	Balance
10001	55000.00
10002	72000.00
10003	120000.00

7 — Bank Money Transfer Using COMMIT



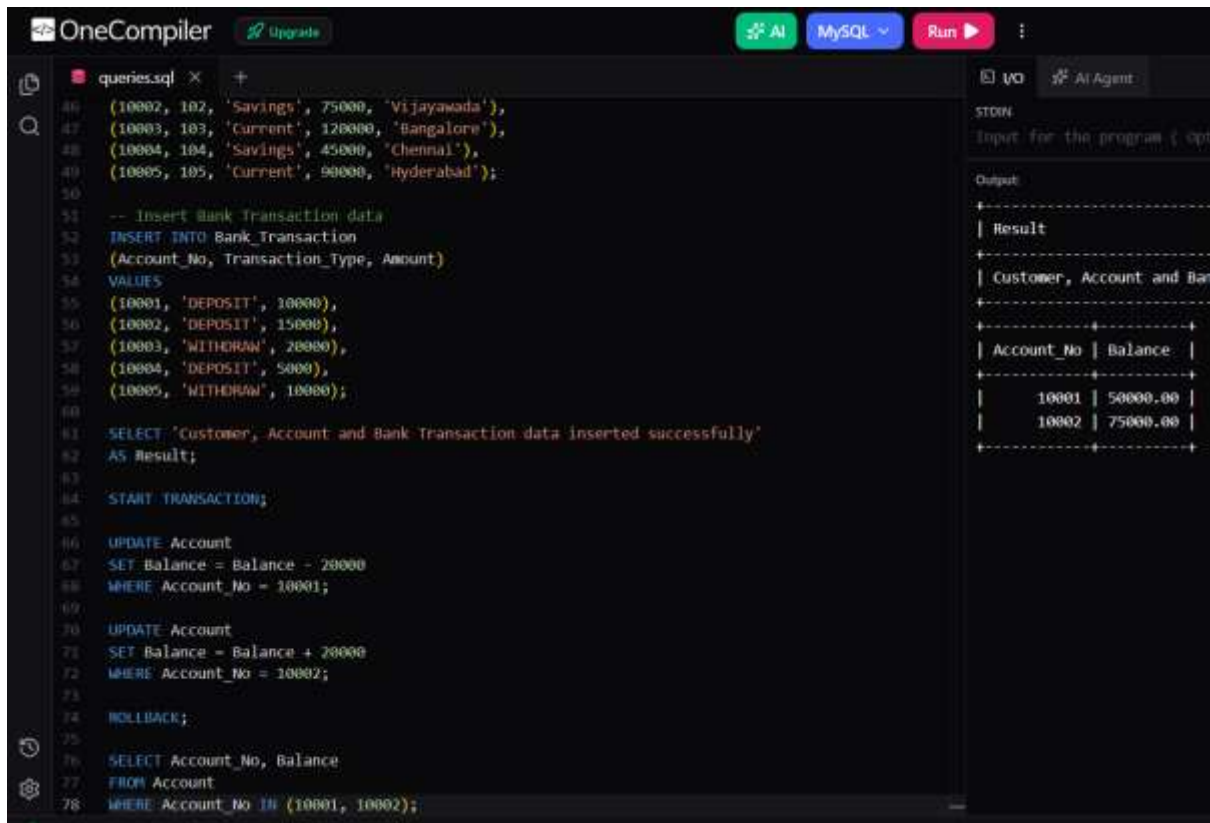
The screenshot shows the OneCompiler MySQL IDE interface. The main editor displays a SQL script in a file named `queries.sql`. The script includes the following SQL statements:

```
46 (10002, 102, 'Savings', 75000, 'Vijayawada'),
47 (10003, 103, 'Current', 120000, 'Bangalore'),
48 (10004, 104, 'Savings', 45000, 'Chennai'),
49 (10005, 105, 'Current', 90000, 'Hyderabad');
50
51 -- Insert Bank Transaction data
52 INSERT INTO Bank_Transaction
53 (Account_No, Transaction_Type, Amount)
54 VALUES
55 (10001, 'DEPOSIT', 10000),
56 (10002, 'DEPOSIT', 15000),
57 (10003, 'WITHDRAW', 20000),
58 (10004, 'DEPOSIT', 5000),
59 (10005, 'WITHDRAW', 10000);
60
61 SELECT 'Customer, Account and Bank Transaction data inserted successfully'
62 AS Result;
63
64 START TRANSACTION;
65
66 UPDATE Account
67 SET Balance = Balance - 10000
68 WHERE Account_No = 10001;
69
70 UPDATE Account
71 SET Balance = Balance + 10000
72 WHERE Account_No = 10002;
73
74 COMMIT;
75
76 SELECT Account_No, Balance
77 FROM Account
78 WHERE Account_No IN (10001, 10002);
```

The right sidebar shows the output of the SQL script. It includes a "STDOUT" section with the text "Input for the program (0)", an "Output" section with a "Result" table, and a "Table" section with a table showing the final account balances.

Account_No	Balance
10001	40000.00
10002	85000.00

8 — Bank Money Transfer Using ROLLBACK



The screenshot shows the OneCompiler MySQL IDE interface. The editor contains a SQL script for a bank money transfer using ROLLBACK. The script includes account data, transaction data, and a transfer process that is rolled back.

```
46 (10002, 102, 'Savings', 75000, 'Vijayawada'),
47 (10003, 103, 'Current', 120000, 'Bangalore'),
48 (10004, 104, 'Savings', 45000, 'Chennai'),
49 (10005, 105, 'Current', 90000, 'Hyderabad');
50
51 -- Insert Bank Transaction data
52 INSERT INTO Bank_Transaction
53 (Account_No, Transaction_Type, Amount)
54 VALUES
55 (10001, 'DEPOSIT', 10000),
56 (10002, 'DEPOSIT', 15000),
57 (10003, 'WITHDRAW', 20000),
58 (10004, 'DEPOSIT', 5000),
59 (10005, 'WITHDRAW', 10000);
60
61 SELECT 'Customer, Account and Bank Transaction data inserted successfully'
62 AS Result;
63
64 START TRANSACTION;
65
66 UPDATE Account
67 SET Balance = Balance - 20000
68 WHERE Account_No = 10001;
69
70 UPDATE Account
71 SET Balance = Balance + 20000
72 WHERE Account_No = 10002;
73
74 ROLLBACK;
75
76 SELECT Account_No, Balance
77 FROM Account
78 WHERE Account_No IN (10001, 10002);
```

The output panel shows the result of the SQL execution:

Output

| Result

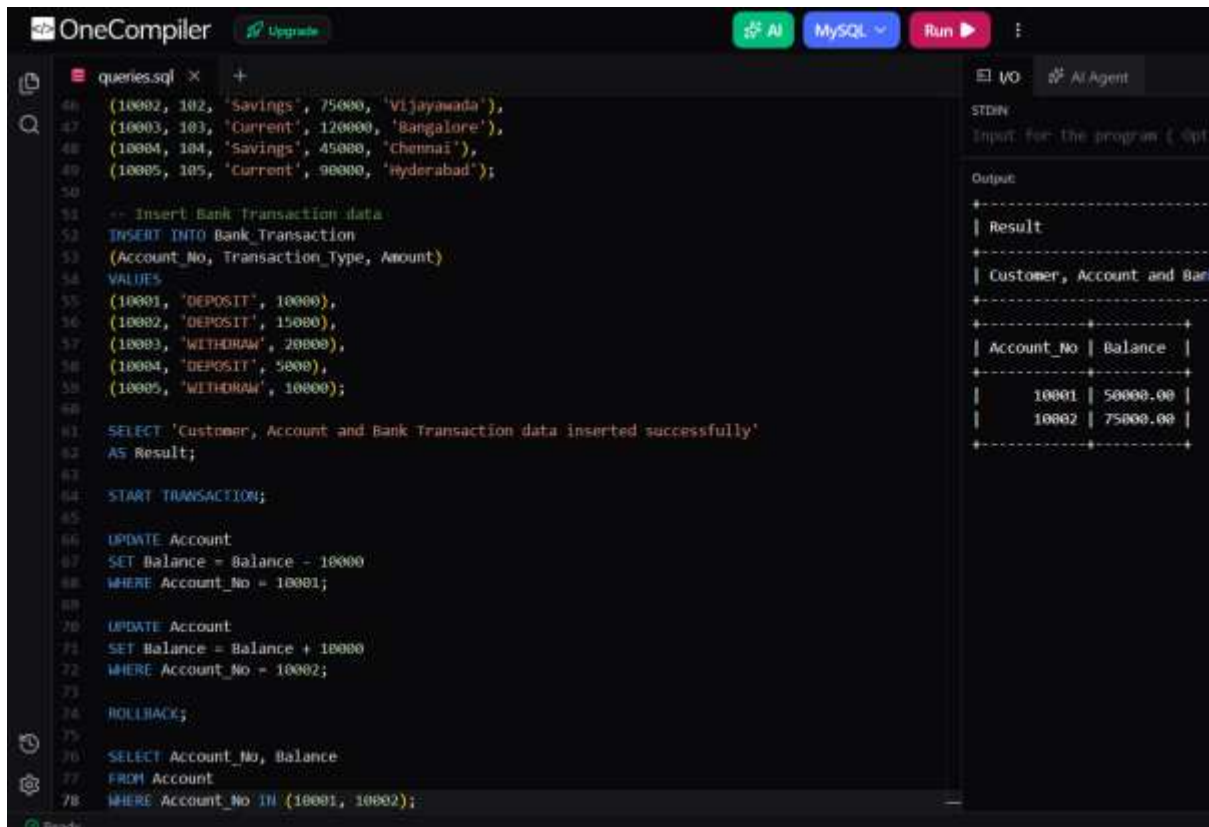
| Customer, Account and Bank Transaction data inserted successfully

| Account_No | Balance |

| 10001 | 50000.00 |

| 10002 | 75000.00 |

9 — ACID: Atomicity



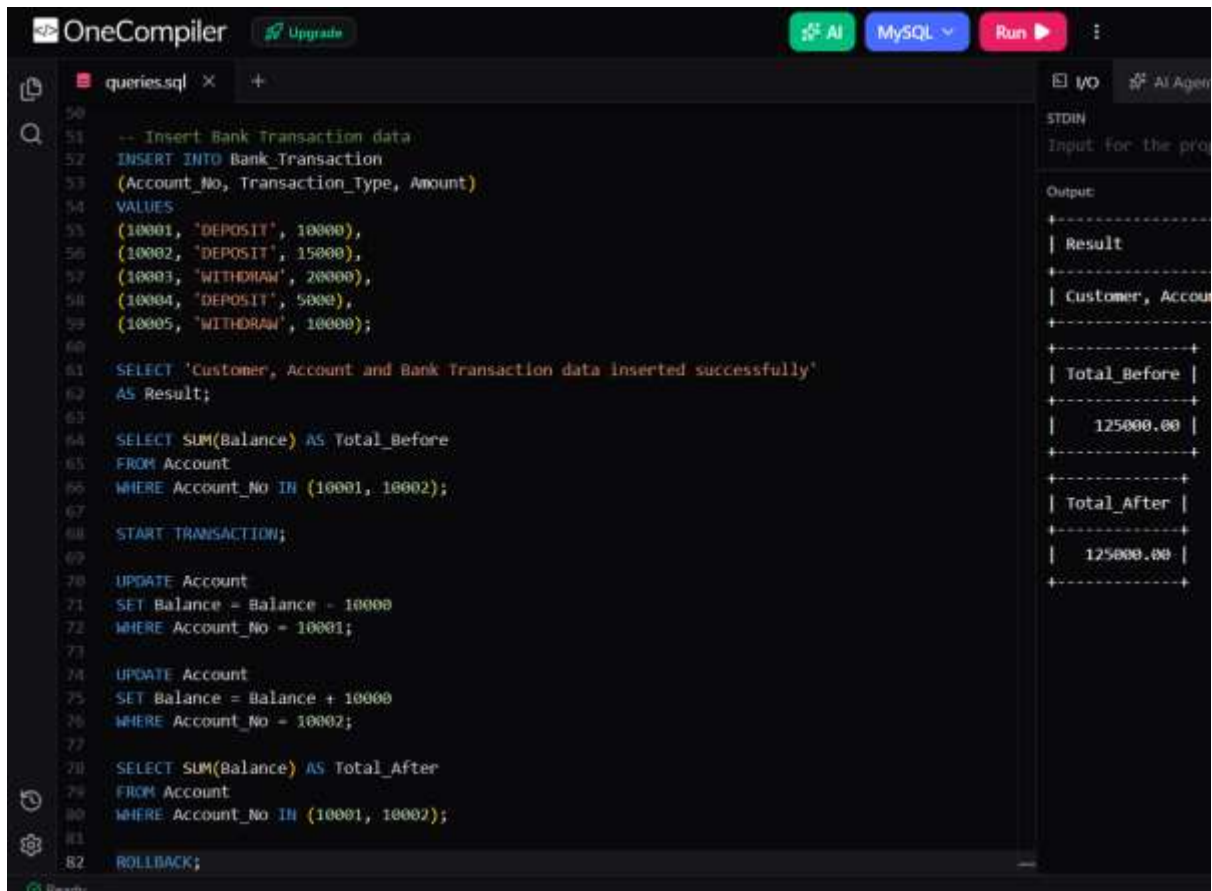
The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL script in a file named `queries.sql`. The script performs the following operations:

- Inserts five rows into the `Bank_Transaction` table with columns `Account_No`, `Transaction_Type`, and `Amount`.
- Starts a transaction with `START TRANSACTION;`.
- Updates the `Account` table, decreasing the balance of account 10001 by 10000.
- Updates the `Account` table, increasing the balance of account 10002 by 10000.
- Rolls back the transaction with `ROLLBACK;`.
- Finally, selects the `Account_No` and `Balance` for accounts 10001 and 10002.

The right pane shows the output of the script. It includes a header "Output" and a section titled "Result" which displays the final state of the `Account` table after the rollback.

Account_No	Balance
10001	50000.00
10002	75000.00

10 — ACID: Consistency



The screenshot shows the OneCompiler MySQL IDE interface. The main editor displays a SQL script in a file named `queries.sql`. The script performs the following operations:

- Inserts five transactions into the `Bank_Transaction` table.
- Selects a success message.
- Calculates the total balance before a transaction for accounts 10001 and 10002, showing a result of 125000.00.
- Starts a transaction.
- Updates the balance of account 10001 by subtracting 10000.
- Updates the balance of account 10002 by adding 10000.
- Calculates the total balance after the transaction, showing a result of 125000.00.
- Rolls back the transaction.

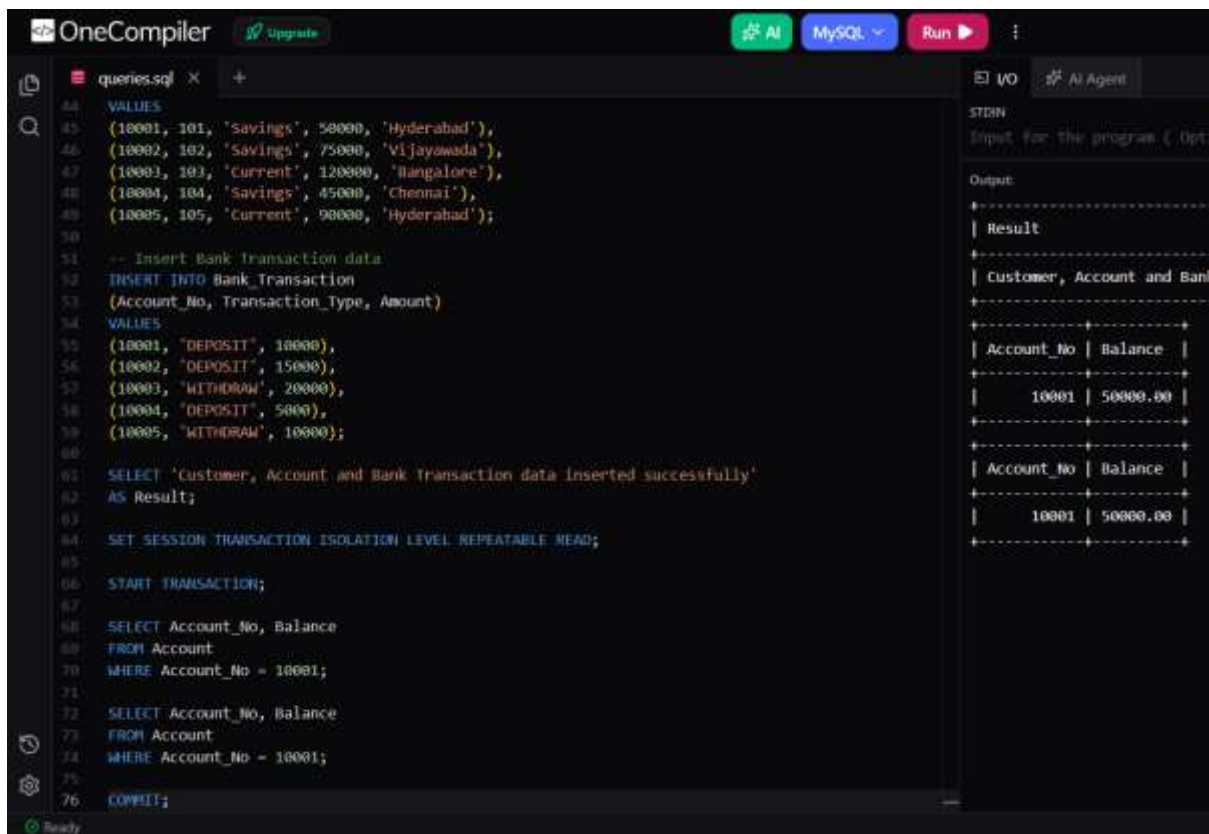
The output panel on the right shows the results of the queries, including the success message and the balance totals before and after the transaction.

```
50
51 -- Insert Bank Transaction data
52 INSERT INTO Bank_Transaction
53 (Account_No, Transaction_Type, Amount)
54 VALUES
55 (10001, 'DEPOSIT', 10000),
56 (10002, 'DEPOSIT', 15000),
57 (10003, 'WITHDRAW', 20000),
58 (10004, 'DEPOSIT', 5000),
59 (10005, 'WITHDRAW', 10000);
60
61 SELECT 'Customer, Account and Bank Transaction data inserted successfully'
62 AS Result;
63
64 SELECT SUM(Balance) AS Total_Before
65 FROM Account
66 WHERE Account_No IN (10001, 10002);
67
68 START TRANSACTION;
69
70 UPDATE Account
71 SET Balance = Balance - 10000
72 WHERE Account_No = 10001;
73
74 UPDATE Account
75 SET Balance = Balance + 10000
76 WHERE Account_No = 10002;
77
78 SELECT SUM(Balance) AS Total_After
79 FROM Account
80 WHERE Account_No IN (10001, 10002);
81
82 ROLLBACK;
```

Output:

```
-----+-----
| Result
+-----+-----
| Customer, Account and Bank Transaction data inserted successfully
+-----+-----
| Total_Before |
+-----+-----
| 125000.00 |
+-----+-----
| Total_After |
+-----+-----
| 125000.00 |
+-----+-----
```

11 — ACID: Isolation



The screenshot shows the OneCompiler MySQL IDE interface. The left pane contains a file named 'queries.sql' with the following SQL code:

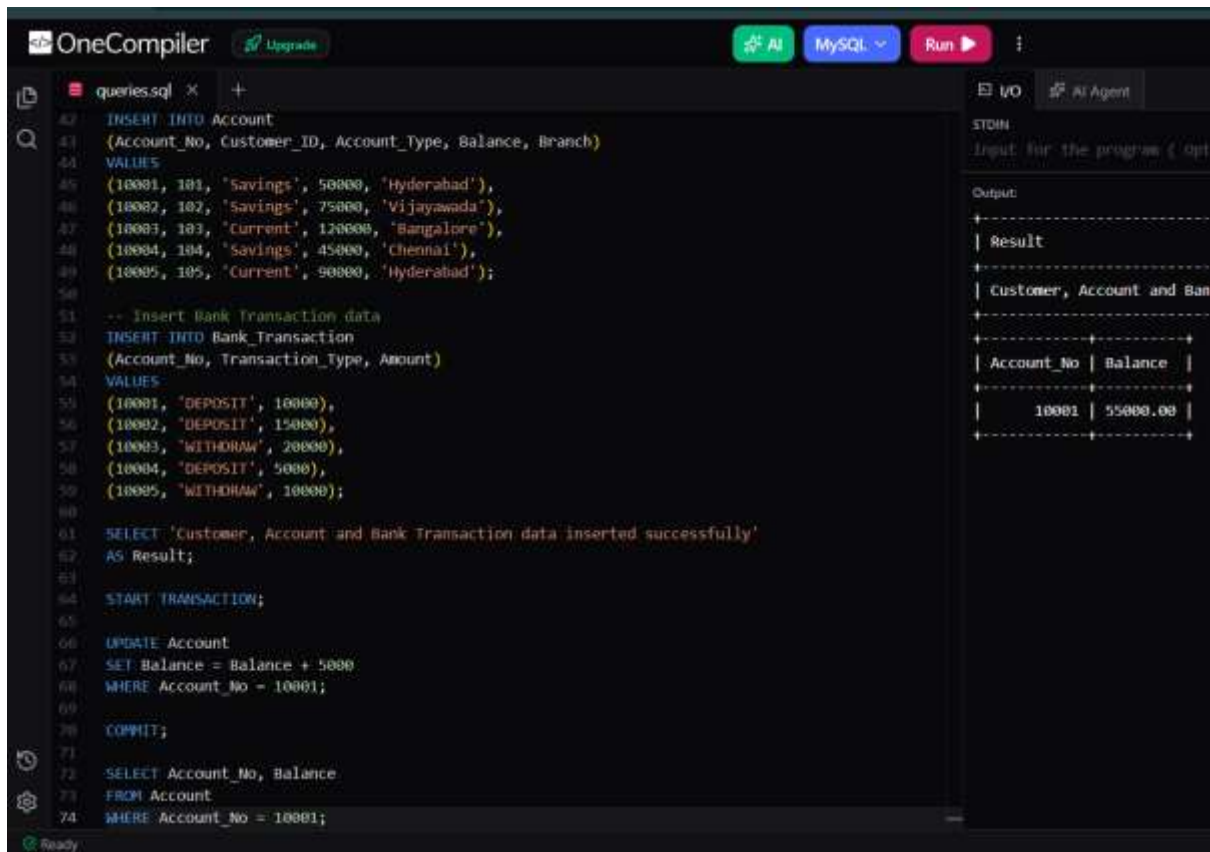
```
44 VALUES
45 (10001, 101, 'Savings', 50000, 'Hyderabad'),
46 (10002, 102, 'Savings', 75000, 'Vijayawada'),
47 (10003, 103, 'Current', 120000, 'Bangalore'),
48 (10004, 104, 'Savings', 45000, 'Chennai'),
49 (10005, 105, 'Current', 90000, 'Hyderabad');
50
51 -- Insert Bank Transaction data
52 INSERT INTO Bank_Transaction
53 (Account_No, Transaction_Type, Amount)
54 VALUES
55 (10001, 'DEPOSIT', 10000),
56 (10002, 'DEPOSIT', 15000),
57 (10003, 'WITHDRAW', 20000),
58 (10004, 'DEPOSIT', 5000),
59 (10005, 'WITHDRAW', 10000);
60
61 SELECT 'Customer, Account and Bank Transaction data inserted successfully'
62 AS Result;
63
64 SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
65
66 START TRANSACTION;
67
68 SELECT Account_No, Balance
69 FROM Account
70 WHERE Account_No = 10001;
71
72 SELECT Account_No, Balance
73 FROM Account
74 WHERE Account_No = 10001;
75
76 COMMIT;
```

The right pane shows the output of the queries. It includes a 'Result' section with a table of account balances and a 'Customer, Account and Bank Transaction data inserted successfully' message.

Account_No	Balance
10001	50000.00
10002	75000.00
10003	120000.00
10004	45000.00
10005	90000.00

The output also includes a message: 'Customer, Account and Bank Transaction data inserted successfully'.

12 — ACID: Durability



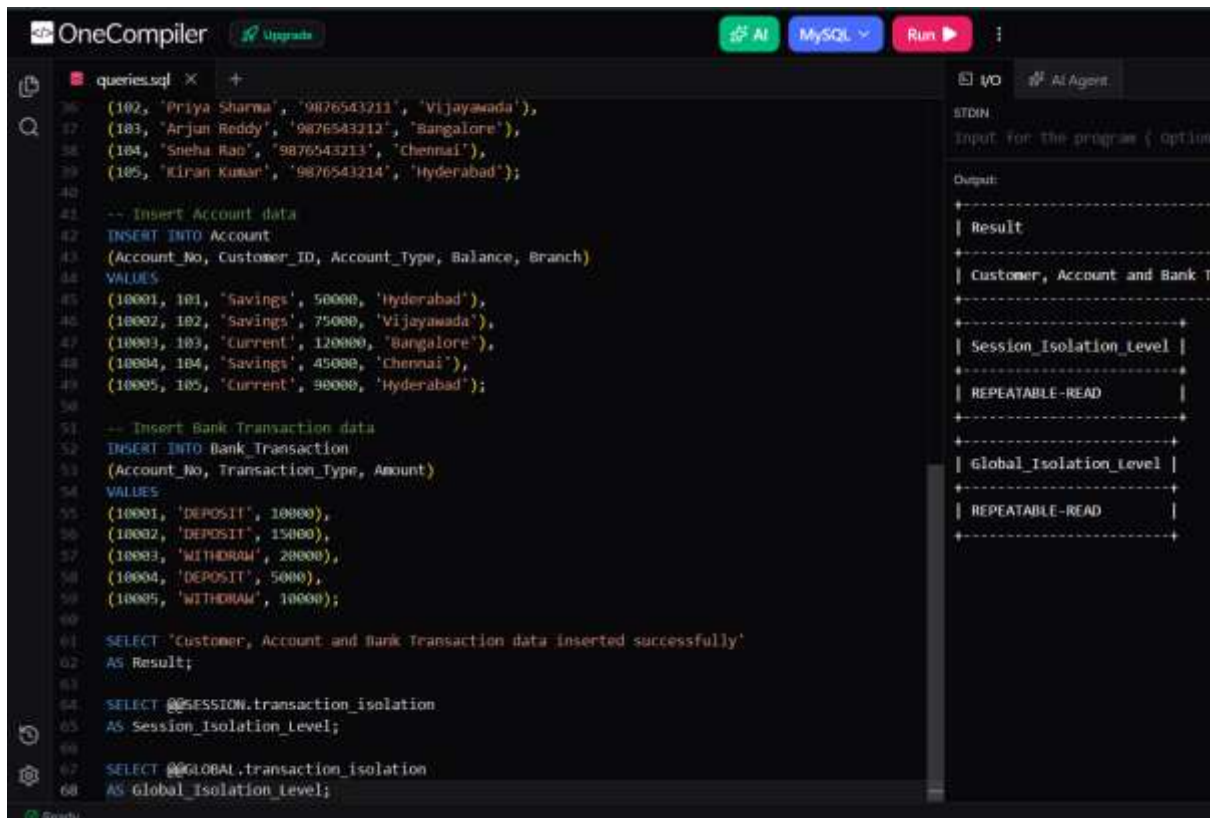
The screenshot shows the OneCompiler MySQL IDE interface. The main editor displays a series of SQL queries in a file named 'queries.sql'. The queries include inserting data into 'Account' and 'Bank_Transaction' tables, starting a transaction, updating the balance of account 10001, and committing the transaction. The output pane on the right shows the results of these queries, including a confirmation message and a table of account balances.

```
42 INSERT INTO Account
43 (Account_No, Customer_ID, Account_Type, Balance, Branch)
44 VALUES
45 (10001, 101, 'Savings', 50000, 'Hyderabad'),
46 (10002, 102, 'Savings', 75000, 'Vijayawada'),
47 (10003, 103, 'Current', 120000, 'Bangalore'),
48 (10004, 104, 'Savings', 45000, 'Chennai'),
49 (10005, 105, 'Current', 90000, 'Hyderabad');
50
51 -- Insert Bank Transaction data
52 INSERT INTO Bank_Transaction
53 (Account_No, Transaction_Type, Amount)
54 VALUES
55 (10001, 'DEPOSIT', 10000),
56 (10002, 'DEPOSIT', 15000),
57 (10003, 'WITHDRAW', 20000),
58 (10004, 'DEPOSIT', 5000),
59 (10005, 'WITHDRAW', 10000);
60
61 SELECT 'Customer, Account and Bank Transaction data inserted successfully'
62 AS Result;
63
64 START TRANSACTION;
65
66 UPDATE Account
67 SET Balance = Balance + 5000
68 WHERE Account_No = 10001;
69
70 COMMIT;
71
72 SELECT Account_No, Balance
73 FROM Account
74 WHERE Account_No = 10001;
```

Output:

```
STDM
Input for the program ( opt
Output
+-----+
| Result
+-----+
| Customer, Account and Ban
+-----+
+-----+-----+
| Account_No | Balance |
+-----+-----+
| 10001      | 55000.00 |
+-----+-----+
```

13 — Check Current Transaction Isolation Level



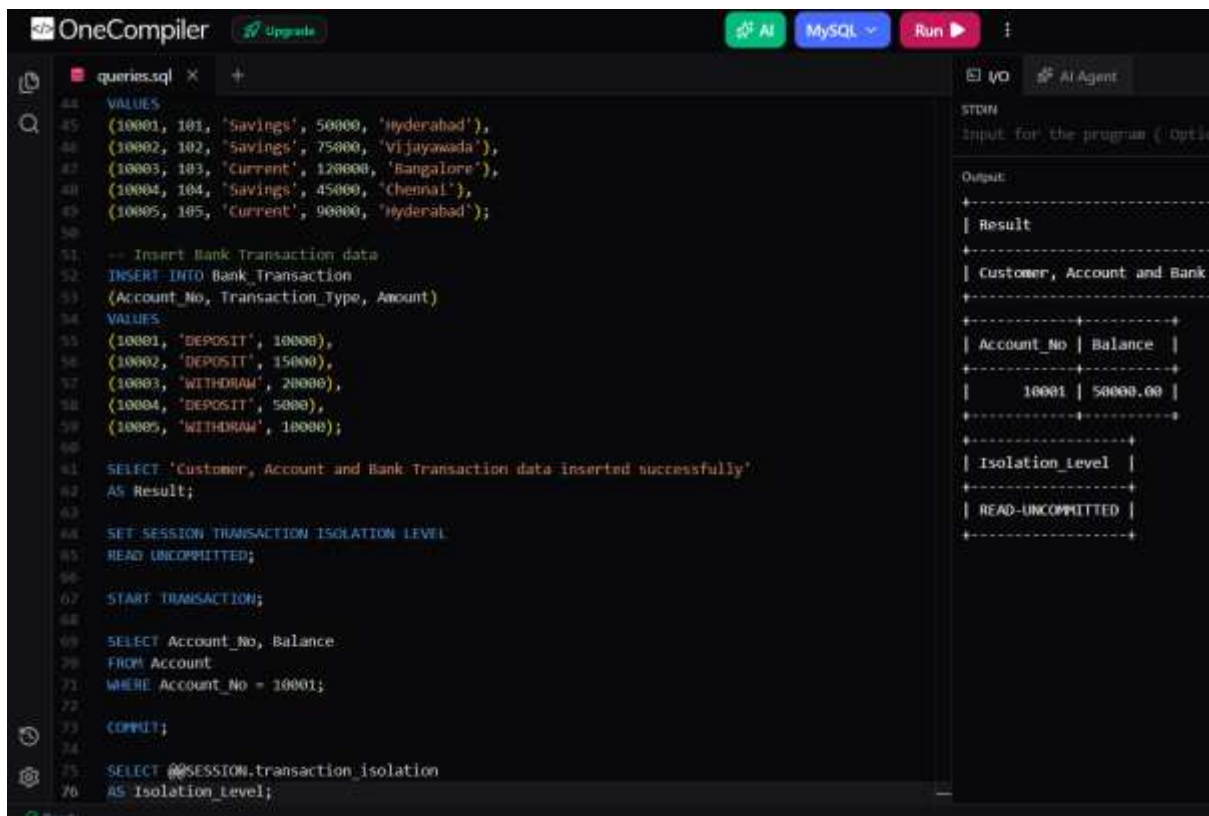
The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL file named 'queries.sql' with the following content:

```
36 (102, 'Priya Sharma', '9876543211', 'Vijayawada'),
37 (103, 'Arjun Reddy', '9876543212', 'Bangalore'),
38 (104, 'Sneha Rao', '9876543213', 'Chennai'),
39 (105, 'Kiran Kumar', '9876543214', 'Hyderabad');
40
41 -- Insert Account data
42 INSERT INTO Account
43 (Account_No, Customer_ID, Account_Type, Balance, Branch)
44 VALUES
45 (10001, 101, 'Savings', 50000, 'Hyderabad'),
46 (10002, 102, 'Savings', 75000, 'Vijayawada'),
47 (10003, 103, 'Current', 120000, 'Bangalore'),
48 (10004, 104, 'Savings', 45000, 'Chennai'),
49 (10005, 105, 'Current', 90000, 'Hyderabad');
50
51 -- Insert Bank Transaction data
52 INSERT INTO Bank_Transaction
53 (Account_No, Transaction_Type, Amount)
54 VALUES
55 (10001, 'DEPOSIT', 10000),
56 (10002, 'DEPOSIT', 15000),
57 (10003, 'WITHDRAW', 20000),
58 (10004, 'DEPOSIT', 5000),
59 (10005, 'WITHDRAW', 10000);
60
61 SELECT 'Customer, Account and Bank Transaction data inserted successfully'
62 AS Result;
63
64 SELECT @@SESSION.transaction_isolation
65 AS Session_Isolation_Level;
66
67 SELECT @@GLOBAL.transaction_isolation
68 AS Global_Isolation_Level;
```

The right pane shows the output of the queries. The first query (line 61) returns the text 'Customer, Account and Bank Transaction data inserted successfully'. The second query (line 64) returns the session isolation level, which is 'REPEATABLE-READ'. The third query (line 67) returns the global isolation level, which is also 'REPEATABLE-READ'.

Result
Customer, Account and Bank Transaction data inserted successfully
Session_Isolation_Level
REPEATABLE-READ
Global_Isolation_Level
REPEATABLE-READ

14 — READ UNCOMMITTED



The screenshot shows the OneCompiler MySQL interface. The left pane contains a file named `queries.sql` with the following SQL code:

```
44 VALUES
45 (10001, 101, 'Savings', 50000, 'Hyderabad'),
46 (10002, 102, 'Savings', 75000, 'Vijayawada'),
47 (10003, 103, 'Current', 120000, 'Bangalore'),
48 (10004, 104, 'Savings', 45000, 'Chennai'),
49 (10005, 105, 'Current', 90000, 'Hyderabad');
50
51 -- Insert Bank Transaction data
52 INSERT INTO Bank_Transaction
53 (Account_No, Transaction_Type, Amount)
54 VALUES
55 (10001, 'DEPOSIT', 10000),
56 (10002, 'DEPOSIT', 15000),
57 (10003, 'WITHDRAW', 20000),
58 (10004, 'DEPOSIT', 5000),
59 (10005, 'WITHDRAW', 10000);
60
61 SELECT 'Customer, Account and Bank Transaction data inserted successfully'
62 AS Result;
63
64 SET SESSION TRANSACTION ISOLATION LEVEL
65 READ UNCOMMITTED;
66
67 START TRANSACTION;
68
69 SELECT Account_No, Balance
70 FROM Account
71 WHERE Account_No = 10001;
72
73 COMMIT;
74
75 SELECT @@SESSION.transaction_isolation
76 AS Isolation_Level;
```

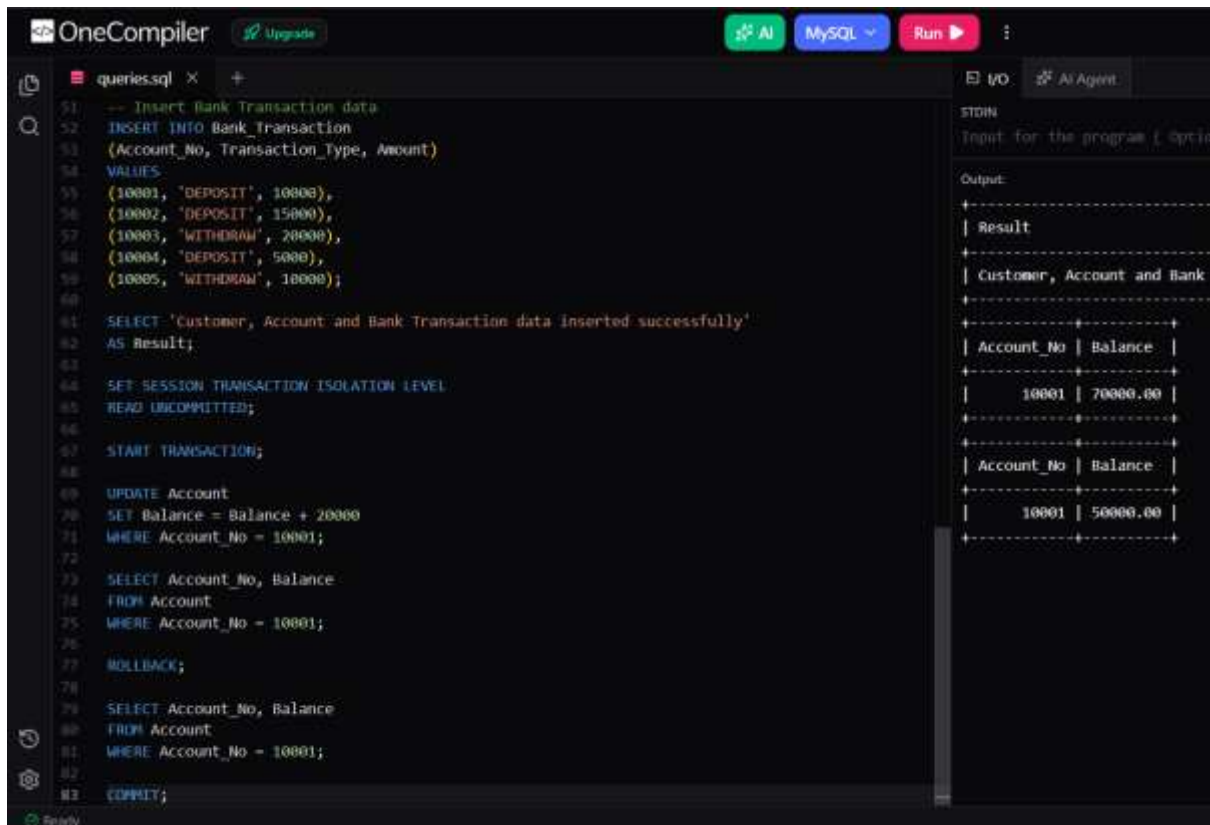
The right pane shows the output of the queries. It includes a section for the first query's result:

Account_No	Balance
10001	50000.00

Below this, the output shows the session's transaction isolation level:

Isolation_Level
READ-UNCOMMITTED

15 — Dirty Read: READ UNCOMMITTED



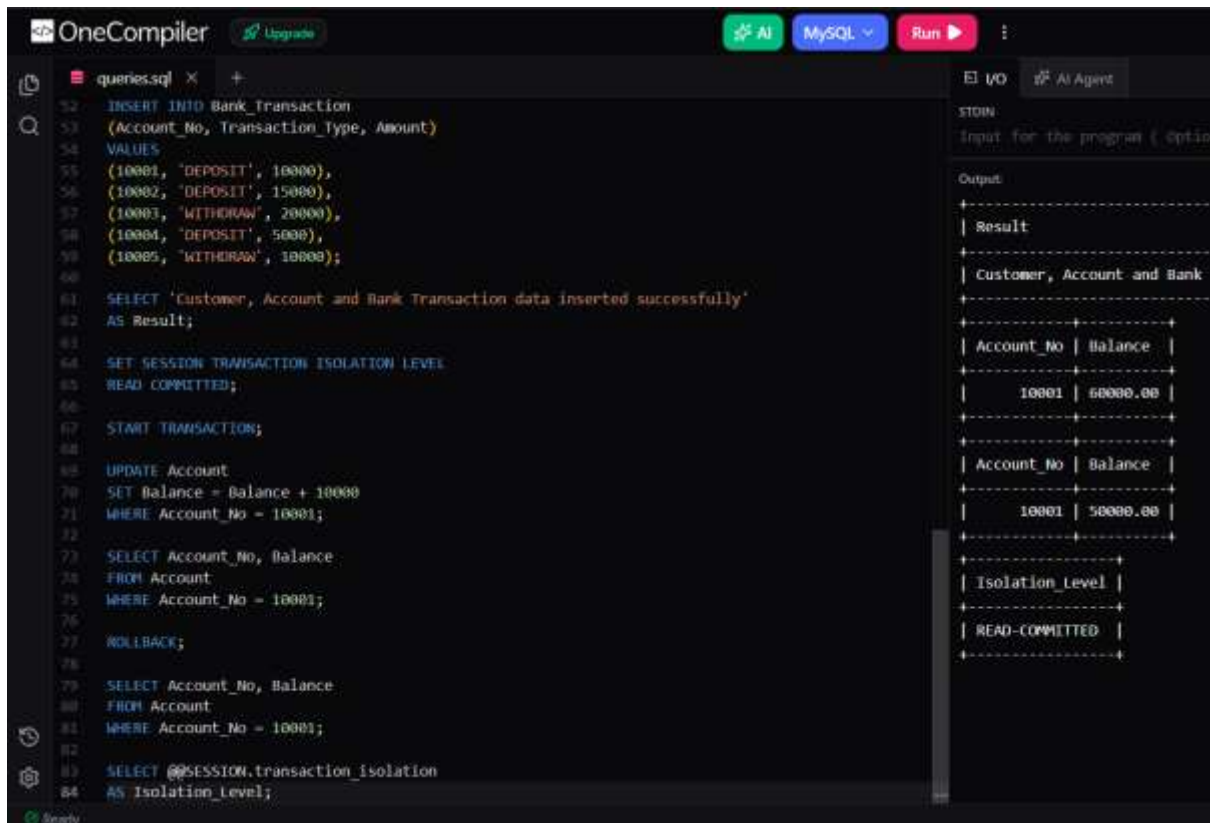
The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL script named 'queries.sql' with the following content:

```
51 -- Insert Bank Transaction data
52 INSERT INTO Bank_Transaction
53 (Account_No, Transaction_Type, Amount)
54 VALUES
55 (10001, 'DEPOSIT', 10000),
56 (10002, 'DEPOSIT', 15000),
57 (10003, 'WITHDRAW', 20000),
58 (10004, 'DEPOSIT', 5000),
59 (10005, 'WITHDRAW', 10000);
60
61 SELECT 'Customer, Account and Bank Transaction data inserted successfully'
62 AS Result;
63
64 SET SESSION TRANSACTION ISOLATION LEVEL
65 READ UNCOMMITTED;
66
67 START TRANSACTION;
68
69 UPDATE Account
70 SET Balance = Balance + 20000
71 WHERE Account_No = 10001;
72
73 SELECT Account_No, Balance
74 FROM Account
75 WHERE Account_No = 10001;
76
77 ROLLBACK;
78
79 SELECT Account_No, Balance
80 FROM Account
81 WHERE Account_No = 10001;
82
83 COMMIT;
```

The right pane shows the output of the script. It includes a 'STDOUT' section with the message 'Input for the program (Optional):' and an 'Output:' section. The output displays two tables. The first table, titled 'Result', shows the message 'Customer, Account and Bank Transaction data inserted successfully'. The second table, titled 'Account_No | Balance |', shows the state of the 'Account' table after the update and rollback. The table has two rows: one with 'Account_No' 10001 and 'Balance' 70000.00, and another with 'Account_No' 10001 and 'Balance' 50000.00.

Account_No	Balance
10001	70000.00
10001	50000.00

16 — READ COMMITTED



The screenshot shows the OneCompiler MySQL interface. The left pane contains a file named `queries.sql` with the following SQL code:

```
52 INSERT INTO Bank_Transaction
53 (Account_No, Transaction_Type, Amount)
54 VALUES
55 (10001, 'DEPOSIT', 10000),
56 (10002, 'DEPOSIT', 15000),
57 (10003, 'WITHDRAW', 20000),
58 (10004, 'DEPOSIT', 5000),
59 (10005, 'WITHDRAW', 10000);
60
61 SELECT 'Customer, Account and Bank Transaction data inserted successfully'
62 AS Result;
63
64 SET SESSION TRANSACTION ISOLATION LEVEL
65 READ COMMITTED;
66
67 START TRANSACTION;
68
69 UPDATE Account
70 SET Balance = Balance + 10000
71 WHERE Account_No = 10001;
72
73 SELECT Account_No, Balance
74 FROM Account
75 WHERE Account_No = 10001;
76
77 ROLLBACK;
78
79 SELECT Account_No, Balance
80 FROM Account
81 WHERE Account_No = 10001;
82
83 SELECT @@SESSION.transaction_isolation
84 AS Isolation_Level;
```

The right pane shows the output of the queries. It includes a section for the result of the first query, followed by two tables showing account balances before and after an update, and finally the session isolation level.

Output

Input for the program (Optional):

Result

Customer, Account and Bank Transaction data inserted successfully

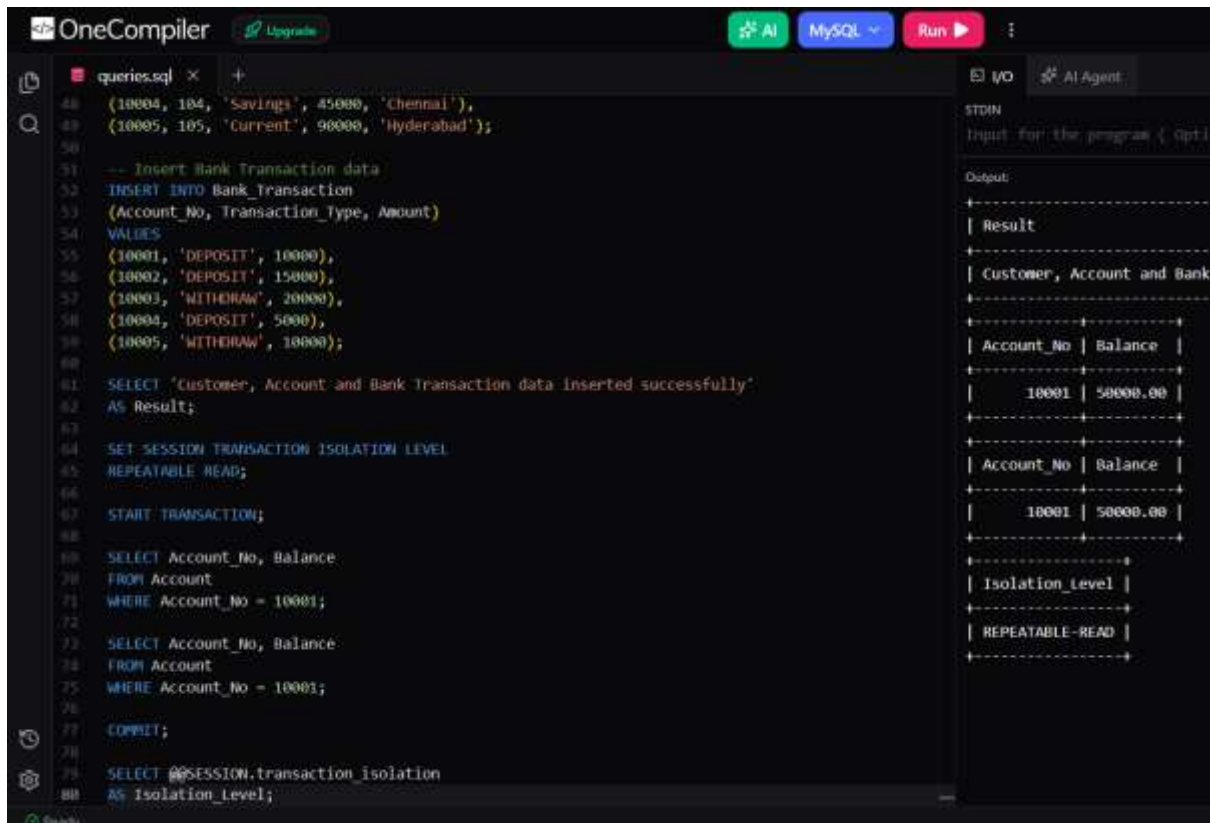
Account_No	Balance
10001	60000.00

Account_No	Balance
10001	50000.00

Isolation_Level

READ-COMMITTED

17 — REPEATABLE READ



The screenshot shows the OneCompiler MySQL interface. The left pane contains a file named `queries.sql` with the following SQL code:

```
48 (10004, 104, 'Savings', 45000, 'Chennai'),
49 (10005, 105, 'Current', 90000, 'Hyderabad');
50
51 -- Insert Bank Transaction data
52 INSERT INTO Bank_Transaction
53 (Account_No, Transaction_Type, Amount)
54 VALUES
55 (10001, 'DEPOSIT', 10000),
56 (10002, 'DEPOSIT', 15000),
57 (10003, 'WITHDRAW', 20000),
58 (10004, 'DEPOSIT', 5000),
59 (10005, 'WITHDRAW', 10000);
60
61 SELECT 'Customer, Account and Bank Transaction data inserted successfully'
62 AS Result;
63
64 SET SESSION TRANSACTION ISOLATION LEVEL
65 REPEATABLE READ;
66
67 START TRANSACTION;
68
69 SELECT Account_No, Balance
70 FROM Account
71 WHERE Account_No = 10001;
72
73 SELECT Account_No, Balance
74 FROM Account
75 WHERE Account_No = 10001;
76
77 COMMIT;
78
79 SELECT @@SESSION.transaction_isolation
80 AS Isolation_Level;
```

The right pane shows the output of the queries. It includes a section for the first query's result:

Account_No	Balance
10001	50000.00

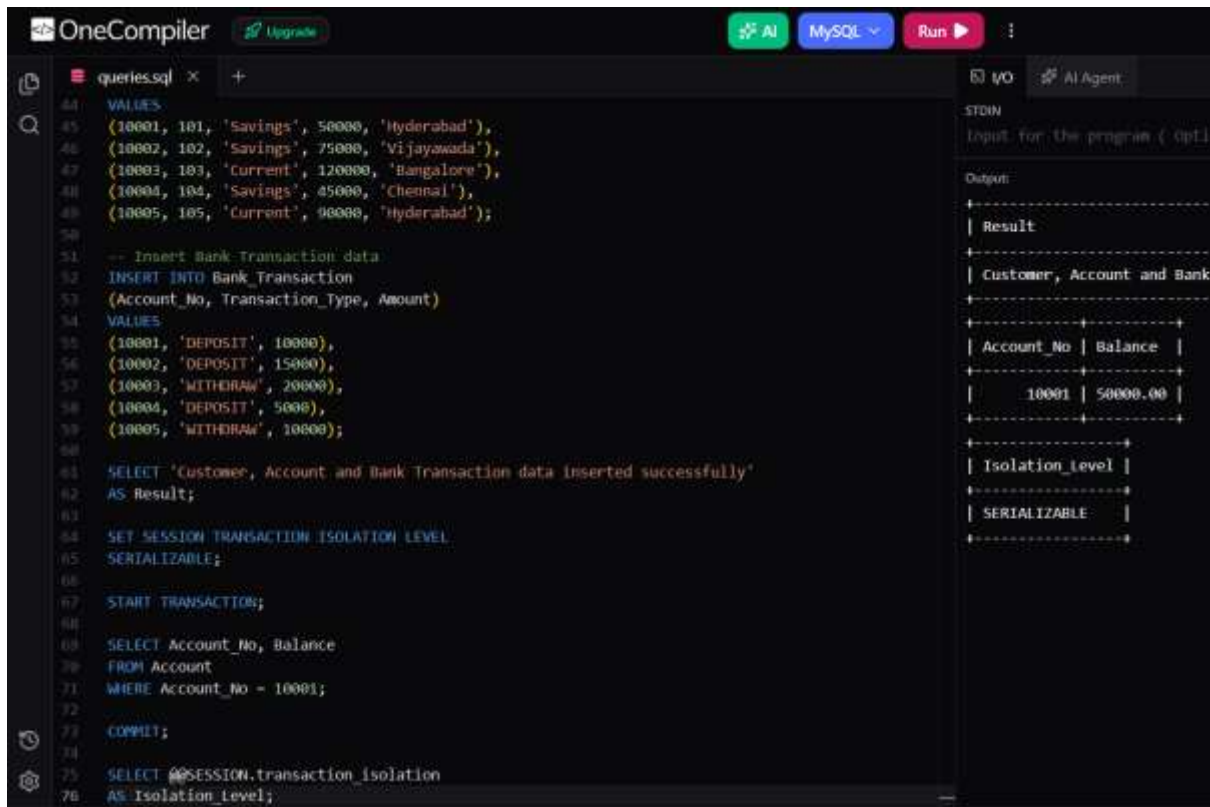
Below this, the output for the `SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;` query is shown:

Account_No	Balance
10001	50000.00

Finally, the output for the `SELECT @@SESSION.transaction_isolation AS Isolation_Level;` query is shown:

Isolation_Level
REPEATABLE-READ

18 — SERIALIZABLE



The screenshot shows the OneCompiler MySQL interface. The left pane contains a file named 'queries.sql' with the following SQL code:

```
44 VALUES
45 (10001, 101, 'Savings', 50000, 'Hyderabad'),
46 (10002, 102, 'Savings', 75000, 'Vijayawada'),
47 (10003, 103, 'Current', 120000, 'Bangalore'),
48 (10004, 104, 'Savings', 45000, 'Chennai'),
49 (10005, 105, 'Current', 90000, 'Hyderabad');
50
51 -- Insert Bank Transaction data
52 INSERT INTO Bank_Transaction
53 (Account_No, Transaction_Type, Amount)
54 VALUES
55 (10001, 'DEPOSIT', 10000),
56 (10002, 'DEPOSIT', 15000),
57 (10003, 'WITHDRAW', 20000),
58 (10004, 'DEPOSIT', 5000),
59 (10005, 'WITHDRAW', 10000);
60
61 SELECT 'Customer, Account and Bank Transaction data inserted successfully'
62 AS Result;
63
64 SET SESSION TRANSACTION ISOLATION LEVEL
65 SERIALIZABLE;
66
67 START TRANSACTION;
68
69 SELECT Account_No, Balance
70 FROM Account
71 WHERE Account_No = 10001;
72
73 COMMIT;
74
75 SELECT @@SESSION.transaction_isolation
76 AS Isolation_Level;
```

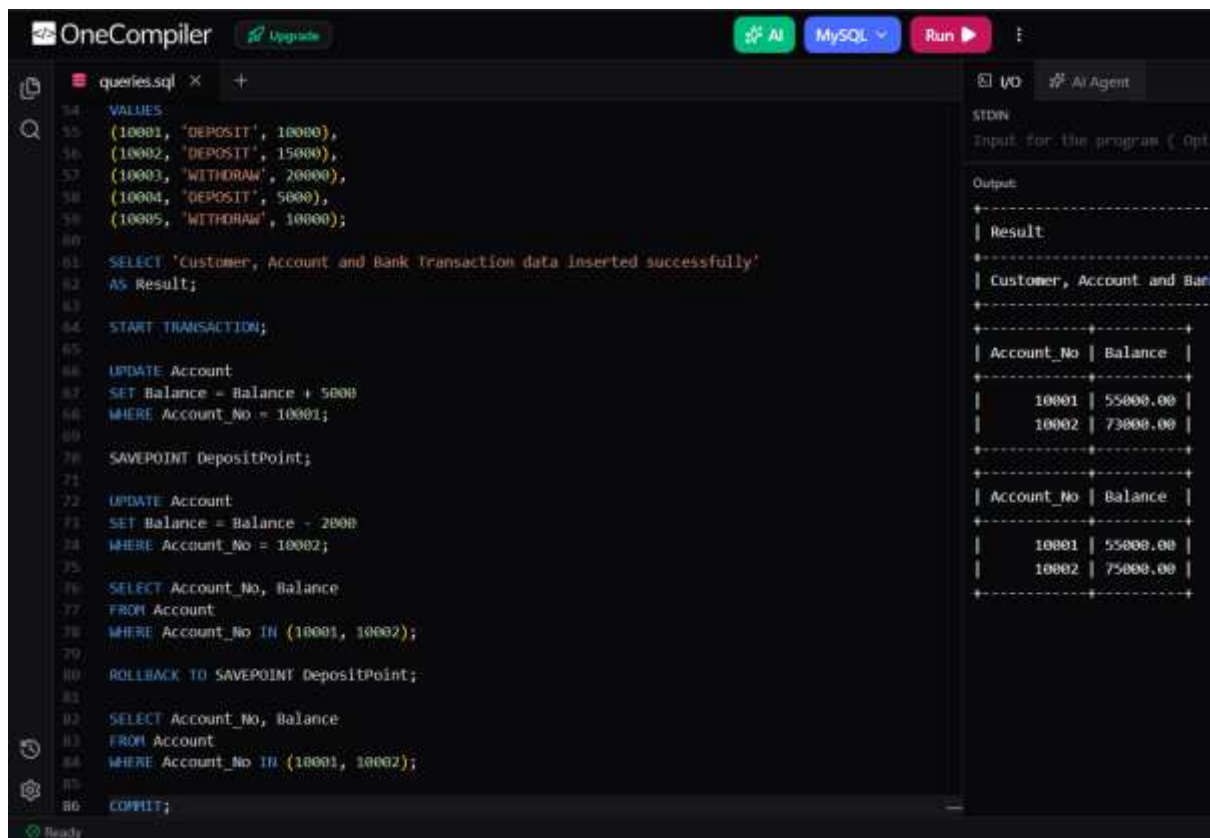
The right pane shows the output of the queries. It includes a 'STDOUT' section with the message 'Input for the program (Opti' and a 'Output:' section. The output displays the result of the first query as a table:

Account_No	Balance
10001	50000.00

Below this, the output shows the result of the second query as a table:

Isolation_Level
SERIALIZABLE

19 — SAVEPOINT AND ROLLBACK TO SAVEPOINT



The screenshot shows the OneCompiler MySQL IDE interface. The editor contains a SQL script with the following lines:

```
14 VALUES
15 (10001, 'DEPOSIT', 10000),
16 (10002, 'DEPOSIT', 15000),
17 (10003, 'WITHDRAW', 20000),
18 (10004, 'DEPOSIT', 5000),
19 (10005, 'WITHDRAW', 10000);
20
21 SELECT 'Customer, Account and Bank Transaction data inserted successfully'
22 AS Result;
23
24 START TRANSACTION;
25
26 UPDATE Account
27 SET Balance = Balance + 5000
28 WHERE Account_No = 10001;
29
30 SAVEPOINT DepositPoint;
31
32 UPDATE Account
33 SET Balance = Balance - 2000
34 WHERE Account_No = 10002;
35
36 SELECT Account_No, Balance
37 FROM Account
38 WHERE Account_No IN (10001, 10002);
39
40 ROLLBACK TO SAVEPOINT DepositPoint;
41
42 SELECT Account_No, Balance
43 FROM Account
44 WHERE Account_No IN (10001, 10002);
45
46 COMMIT;
```

The output panel on the right shows the results of the execution:

Output

Result

Customer, Account and Bank Transaction data inserted successfully

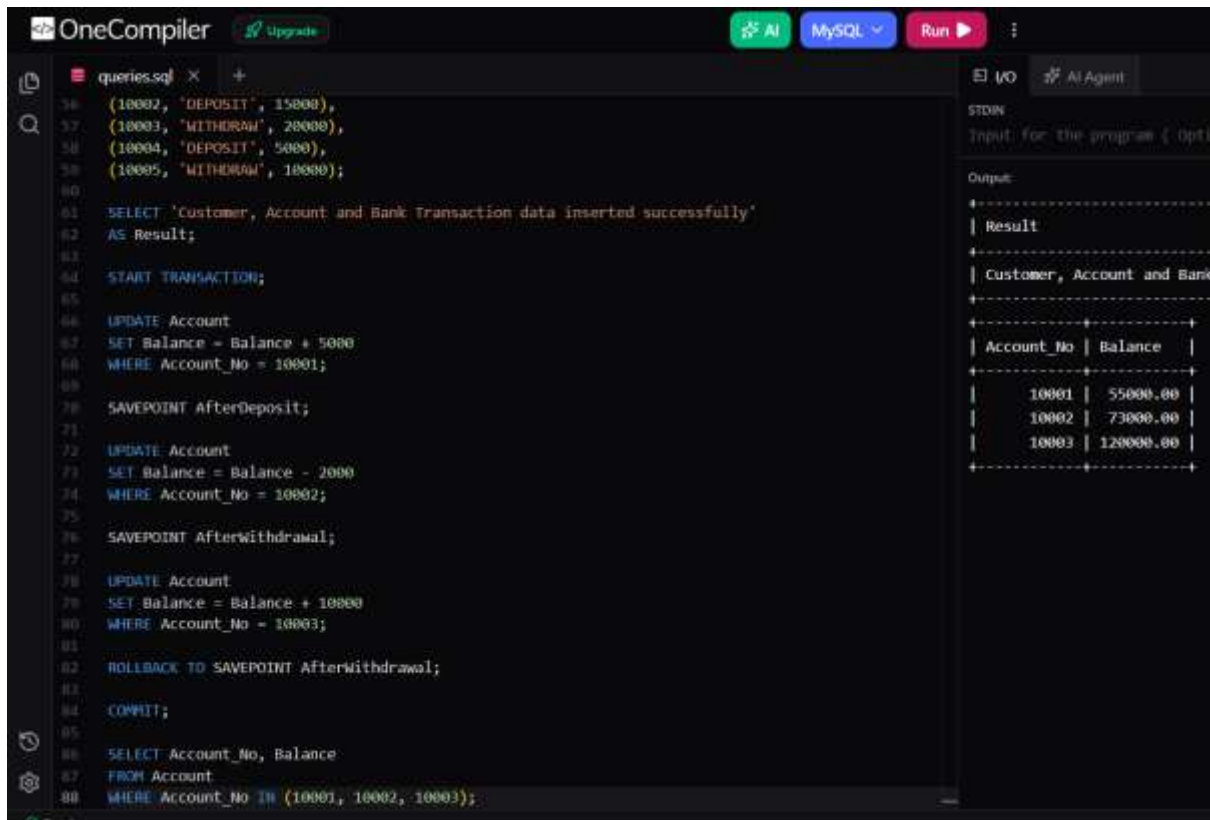
Account_No	Balance
10001	55000.00
10002	73000.00

Account_No | Balance

Account_No	Balance
10001	55000.00
10002	75000.00

Account_No | Balance

20 — Bank Transactions Using SAVEPOINT



The screenshot shows the OneCompiler MySQL IDE. The left pane displays a SQL script in a file named 'queries.sql'. The script performs several database operations: inserting transaction data, committing, updating account balances, setting savepoints, rolling back to a specific savepoint, committing again, and finally selecting the account balances. The right pane shows the output of the script, which includes a confirmation message and a table of account balances.

```
16 (10002, 'DEPOSIT', 15000),
17 (10003, 'WITHDRAW', 20000),
18 (10004, 'DEPOSIT', 5000),
19 (10005, 'WITHDRAW', 10000);
20
21 SELECT 'Customer, Account and Bank Transaction data inserted successfully'
22 AS Result;
23
24 START TRANSACTION;
25
26 UPDATE Account
27 SET Balance = Balance + 5000
28 WHERE Account_No = 10001;
29
30 SAVEPOINT AfterDeposit;
31
32 UPDATE Account
33 SET Balance = Balance - 2000
34 WHERE Account_No = 10002;
35
36 SAVEPOINT AfterWithdrawal;
37
38 UPDATE Account
39 SET Balance = Balance + 10000
40 WHERE Account_No = 10003;
41
42 ROLLBACK TO SAVEPOINT AfterWithdrawal;
43
44 COMMIT;
45
46 SELECT Account_No, Balance
47 FROM Account
48 WHERE Account_No IN (10001, 10002, 10003);
```

Output:

STDIN
Input: for the program (Optional):

Result

Customer, Account and Bank Transaction data inserted successfully

Account_No	Balance
10001	55000.00
10002	73000.00
10003	120000.00